Experiment – 4

Development of RESTful APIs using Node.js and Express.js and Testing using Postman

---

Name:

Class:

Roll No.:

Date:

Aim

To develop and test RESTful APIs using **Node.js and Express.js** and understand HTTP methods, request and response handling, routing, status codes, and API testing using Postman.

Problem Statement

Develop a RESTful API for an Event Management System using Node.js and Express.js.

The API should allow users to perform CRUD (Create, Read, Update, Delete) operations on event information. The developed APIs must be tested using **Postman** to verify their functionality, request parameters, responses, and HTTP status codes.

The application should demonstrate the following concepts:

- Understanding REST architecture and RESTful APIs.
- Creating a backend server using Node.js and Express.js.
- Defining API routes and handling HTTP requests.
- Implementing CRUD operations on event data.
- Using HTTP methods such as GET, POST, PUT, and DELETE.
- Handling request bodies and route parameters.
- Sending appropriate HTTP status codes and JSON responses.
- Testing RESTful APIs using Postman.
- Integrating the API with a frontend application

The same concepts may also be demonstrated using the student's own project topic.

Theory Questions

1. What is an API?
2. What is a RESTful API?
3. What are the main principles of REST architecture?
4. What are HTTP methods? Explain GET, POST, PUT, and DELETE.
5. What is the purpose of Express.js in backend development?
6. What is routing in Express.js?
7. What is the purpose of `req` and `res` in Express.js?
8. What is the purpose of `req.params` and `req.body`?
9. What is JSON, and why is it commonly used in APIs?
10. What are HTTP status codes? Explain 200, 201, 400, 404, and 500.
11. What is Postman, and why is it used for API testing?
12. What is the purpose of the request body and headers in Postman?
13. What is CRUD, and how is it implemented using RESTful APIs?
14. What is middleware in Express.js?

Code

Add important code snippets demonstrating:

1. Creating an Express server.
2. Configuring middleware using `express.json()`.
3. Defining RESTful API routes.
4. Implementing GET operations to retrieve events.
5. Implementing POST operations to create an event.
6. Implementing PUT operations to update an event.
7. Implementing DELETE operations to remove an event.
8. Accessing route parameters using `req.params`.
9. Accessing request data using `req.body`.
10. Sending JSON responses using `res.json()`.
11. Handling errors and returning appropriate HTTP status codes.

Screenshots

A. Attach screenshots demonstrating the following:
    1. GET request to retrieve all events.
    2. GET request to retrieve an event by ID.
    3. POST request to create a new event.
    4. PUT request to update an event.
    5. DELETE request to remove an event.
    6. An invalid request demonstrating error handling.
B. Attach screenshots of the frontend that fetches data from Server and displays.

For each screenshot, mention the HTTP method, endpoint, status code, and expected result.

Result

The RESTful APIs for the Event Management System were developed using Node.js and Express.js and tested using Postman. The functionality of the CRUD operations and API response handling was verified.

Repository Link: _____

Postman Collection Link (optional): _____